```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_excel('/bank_data.xlsx')
```

```
df.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	da
0	28	blue-collar	married	secondary	no	723	yes	yes	unknown	1900 01-0 00:00:0
1	59	blue-collar	married	secondary	no	0	yes	no	unknown	1900 01-0 00:00:0
2	25	blue-collar	married	secondary	no	-7	yes	no	unknown	1900 01-0 00:00:0
3	54	blue-collar	married	secondary	no	1291	yes	no	unknown	1900 01-0 00:00:0
4	57	blue-collar	married	secondary	no	5935	yes	yes	unknown	1900 01-0 00:00:0

5 rows × 21 columns

```
df['housing'].unique()
```

```
array(['yes', 'no'], dtype=object)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 21 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   age          45211 non-null  int64
 1   job          45211 non-null  object
 2   marital      45211 non-null  object
 3   education    45211 non-null  object
 4   default      45211 non-null  object
 5   balance      45211 non-null  int64
 6   housing      45211 non-null  object
 7   loan         45211 non-null  object
 8   contact      45211 non-null  object
 9   day          45211 non-null  object
 10  month        45211 non-null  object
```

```
 11  duration     45211 non-null  object
 12  campaign     45211 non-null  int64
 13  pdays        45211 non-null  int64
 14  previous     45211 non-null  int64
 15  poutcome     45211 non-null  object
 16  y            45211 non-null  object
 17  loan stats   45211 non-null  int64
 18  Loan_period  45211 non-null  int64
 19  CREDIT CARD  45211 non-null  int64
 20  DEBIT CARD   45211 non-null  int64
dtypes: int64(9), object(12)
memory usage: 7.2+ MB
```

```
df.describe()
```

	age	balance	campaign	pdays	previous	loan
count	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.0
mean	40.936210	1362.272058	2.763841	40.197828	0.580323	0.0
std	10.618762	3044.765829	3.098021	100.128746	2.303441	0.1
min	18.000000	-8019.000000	1.000000	-1.000000	0.000000	0.0
25%	33.000000	72.000000	1.000000	-1.000000	0.000000	0.0
50%	39.000000	448.000000	2.000000	-1.000000	0.000000	0.0
75%	48.000000	1428.000000	3.000000	-1.000000	0.000000	0.0
max	95.000000	102127.000000	63.000000	871.000000	275.000000	1.0

```
print(f"Dataset Shape: {df}")

print(df.dtypes)
```

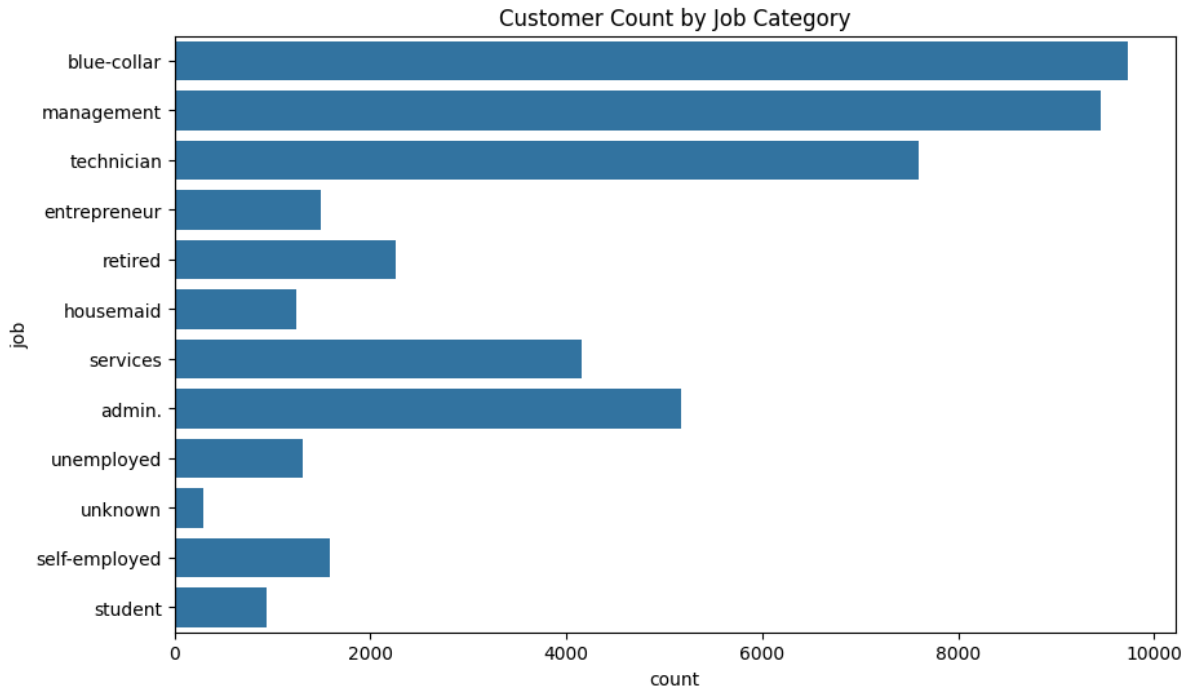
Show hidden output

```
avg_mean=df.groupby('job')['balance'].mean().round(2).sort_values(ascending=False
pd.DataFrame(avg_mean)
```

Show hidden output

Customers occupation by count using countplot

```
plt.figure(figsize=(10,6))
sns.countplot(y='job', data=df, order=df['job'])
plt.title('Customer Count by Job Category')
plt.show()
```




```
!pip uninstall -y pandas-profiling
!pip install ydata-profiling
```

```
from pandas_profiling import ProfileReport
f=ProfileReport(df)
pd.to_file(output='output.html')
```

```
df.sample(5)
```

	age	job	marital	education	default	balance	housing	loan	cont
42645	42	management	married	tertiary	no	1205	no	no	teleph
28787	33	technician	single	tertiary	no	3255	no	no	teleph
958	40	blue-collar	divorced	primary	no	737	yes	yes	unkno
687	53	blue-collar	married	unknown	no	2785	yes	no	unkno
20908	31	services	single	secondary	no	31	yes	yes	unkno

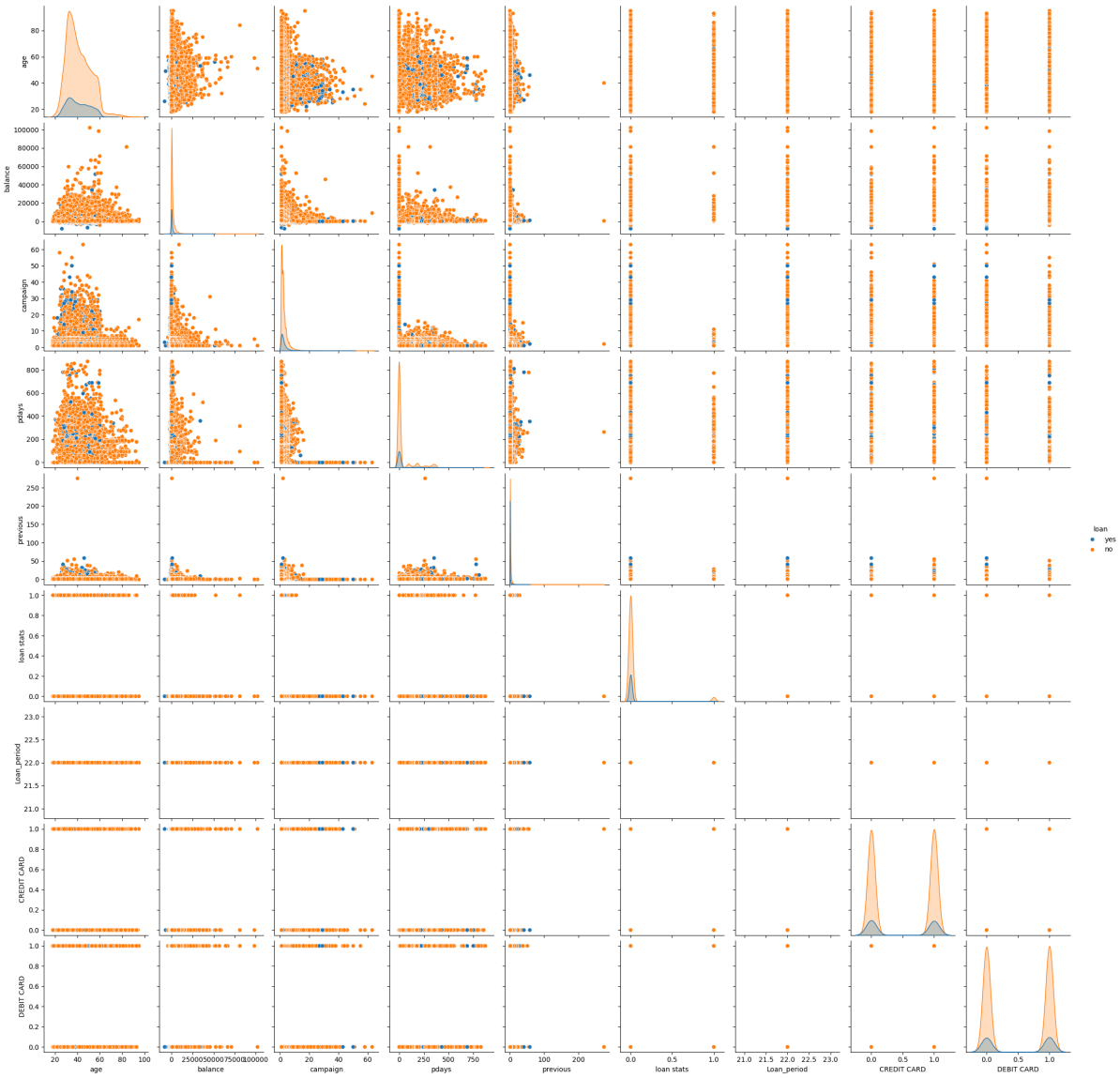
5 rows × 21 columns

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
sns.pairplot(df,hue='loan')
```

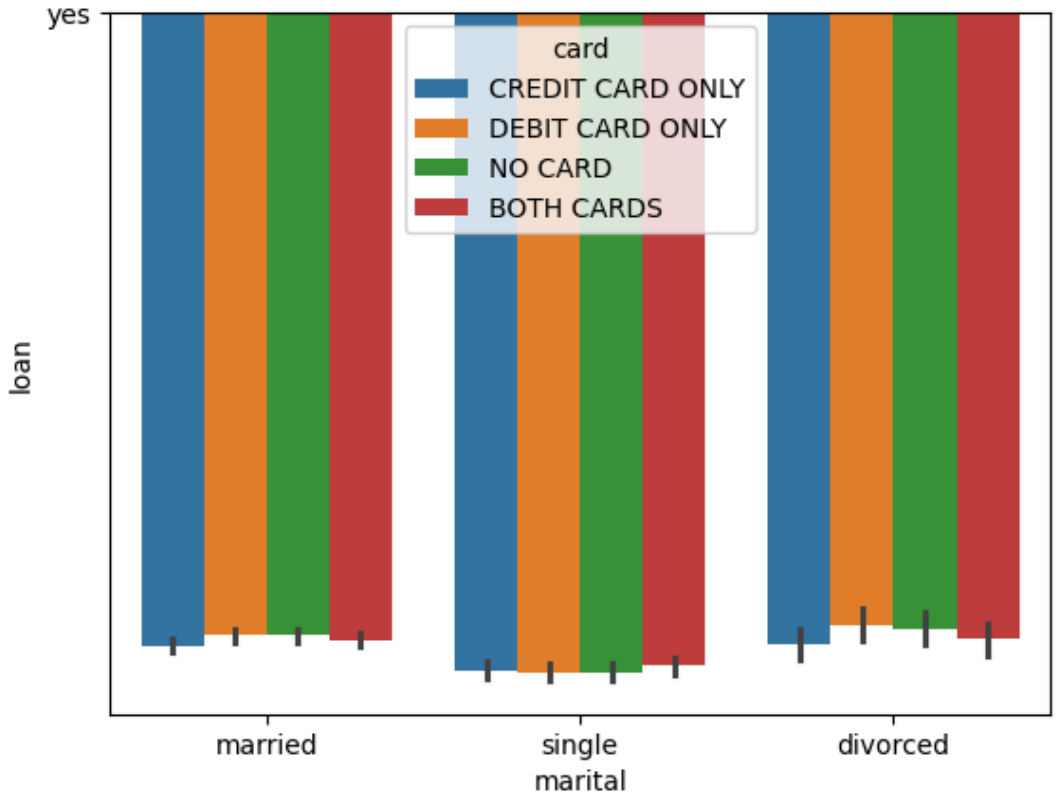
```
<seaborn.axisgrid.PairGrid at 0x7bb375cbd460>
```



Checking which customer with ongoing loan has which type of card

```
df['card'] = df['CREDIT CARD'].astype(str) + df['DEBIT CARD'].astype(str)
df['card']=df['card'].map({'11':'BOTH CARDS', '10':'CREDIT CARD ONLY',
 '01':'DEBIT CARD ONLY', '00':'NO CARD'})
sns.barplot(x='marital',y='loan',hue='card',data=df)
```

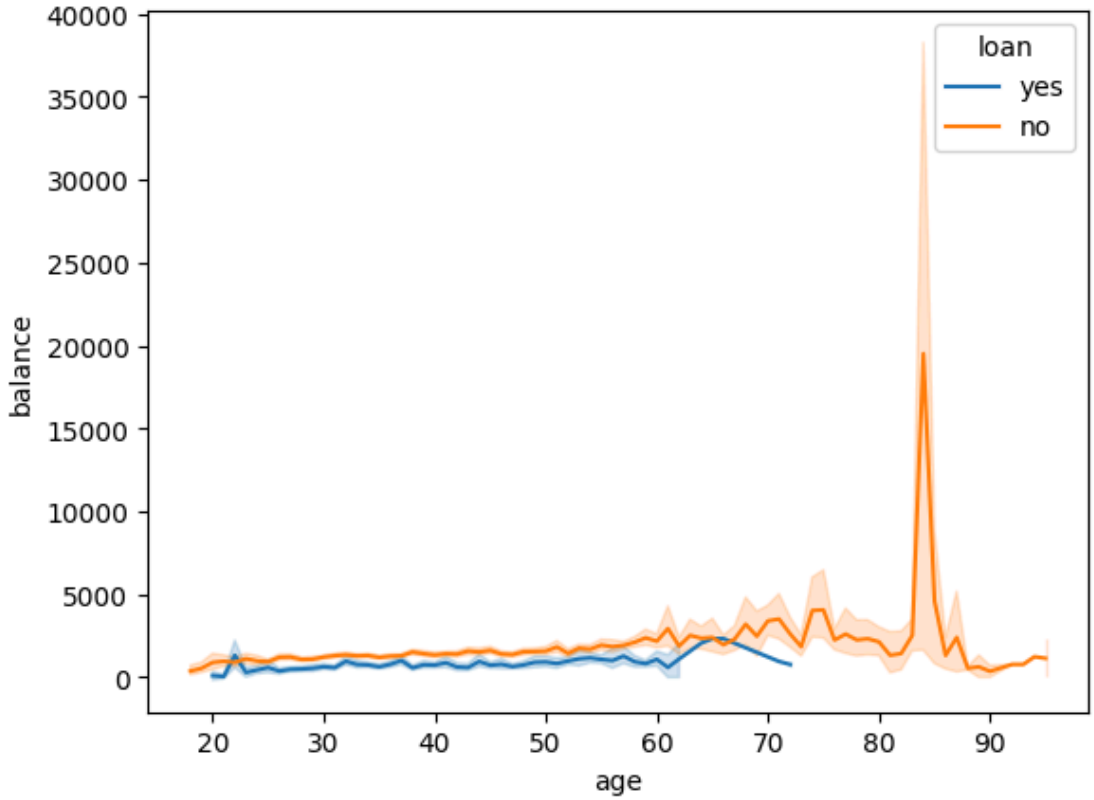
```
<Axes: xlabel='marital', ylabel='loan'>
```



Plotting age vs balance to see which customer segemnt has more deposit in the bank's account along with ongoing loan.

```
sns.lineplot(x='age',y='balance',hue='loan',data=df)
```

```
<Axes: xlabel='age', ylabel='balance'>
```



Before Mapping strings to numeric type for classififcation

```
df_dropped_default = df.drop(columns=['default'])
df1 = df_dropped_default.iloc[:, 0:7]
df1
```

Mapped the housing & loan to numeric values for further classification without using OneHotEncoding/Ordinal_encoding

```
df1['marital']=df1['marital'].map({'married':2,'single':1,'divorced':0})
df1['housing']=df1['housing'].map({'yes':1,'no':0})
df1['loan']=df1['loan'].map({'yes':1,'no':0})
df1=df1.drop(columns=['education','job'],axis=1)
df1.sample(3)
```

	age	marital	balance	housing	loan
2783	56	2	28	1	0
23620	40	0	0	0	0
7073	61	0	52587	0	0

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(df1.drop('loan',axis=1),df1['loan'
x_train
```

	age	marital	balance	housing
3344	57	1	3551	1
17965	26	2	1	1
18299	47	2	846	1
10221	57	2	256	0
32192	47	2	12276	0
...	...	...	...	...
11284	30	2	1930	0
44732	39	2	0	1
38158	35	2	1257	1
860	26	1	309	1
15795	33	1	138	0

36168 rows × 4 columns

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(x_train)
```

```
sc.transform(x_train)
sc.transform(x_test)
```

```
array([[-0.27498346,  0.74098173, -0.1353328 ,  0.8946943 ],
       [ 0.76138965,  0.74098173, -0.32257733, -1.1177002 ],
       [ 0.85560539,  0.74098173, -0.33250437,  0.8946943 ],
       ...,
       [ 0.76138965,  0.74098173, -0.19386811,  0.8946943 ],
       [-0.36919919,  0.74098173, -0.40233596,  0.8946943 ],
       [-0.84027788,  0.74098173, -0.43143246,  0.8946943 ]])
```

ML models without using Encoding methods:

1.Logistic_Regression

```
from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()
lr.fit(x_train, y_train)
print('Logistic_Regression predicted training score',lr.score(x_train,y_train)*10
print('Logistic_Regression predicted test score',lr.score(x_test,y_test)*100)
```

```
Logistic_Regression predicted training score 83.96372483963724
Logistic_Regression predicted test score 84.03184783810683
```

2.XGBoost

```
from xgboost import XGBClassifier
xg=XGBClassifier()
xg=xg.fit(x_train,y_train)
print('XGBoost predicted training score',xg.score(x_train,y_train)*100)
print('XGBoost predicted test score',xg.score(x_test,y_test)*100)
```

```
XGBoost predicted training score 84.60517584605176
XGBoost predicted test score 83.86597368130046
```

3.Random_Forest

```
from sklearn.ensemble import RandomForestClassifier
rf=RandomForestClassifier(n_estimators=150,max_depth=5)
rf=rf.fit(x_train,y_train)
print('Random Forest predicted training score',rf.score(x_train,y_train)*100)
print('Random Forest predicted test score',rf.score(x_test,y_test)*100)
```

```
Random Forest predicted training score 84.08814421588144
Random Forest predicted test score 84.04290611522724
```

Using Encoding techniques

```
df2 = df.drop(df.iloc[:, 8:15].columns, axis=1)
df2.sample(3)
```

	age	job	marital	education	default	balance	housing	loan	poutcome
20722	35	technician	single	secondary	no	433	yes	no	unknown
35332	38	self-employed	single	tertiary	no	605	no	no	unknown
17361	32	blue-collar	divorced	secondary	no	-299	yes	no	unknown

Used Column Transformer for using different encoding techniques. ^OneHotEncoder ^OrdinalEncoder

```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer

ct=ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(drop='first', sparse_output=False),
         ['job','marital', 'poutcome', 'card']),
        ('ordinal', OrdinalEncoder(categories=[
            ['unknown','primary','secondary','tertiary'],
            ['no','yes'],
            ['no','yes'],
            ['no','yes']
        ]), ['education','housing', 'default', 'y'])
      ],
    remainder='passthrough'
)
```

Splitting the data as training and test data for further calculations. Mapping the string values to numeric

```
from sklearn.model_selection import train_test_split
X_train,X_test,Y_train,Y_test=train_test_split(df2.drop(columns='loan',axis=1),df
Y_train = Y_train.map({'yes': 1, 'no': 0})
Y_test = Y_test.map({'yes': 1, 'no': 0})
X_train = ct.fit_transform(X_train)
X_test = ct.transform(X_test)
```

ML Model using Column_Transform

1.LogisticRegression

```
from sklearn.linear_model import LogisticRegression
```

```
l=LogisticRegression()
l=l.fit(X_train, Y_train)
print('Logistic_Regression predicted training score',l.score(X_train,Y_train)*100)
print('Logistic_Regression predicted test score',l.score(X_test,Y_test)*100)
```

```
Logistic_Regression predicted training score 83.96648971466489
Logistic_Regression predicted test score 84.03184783810683
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: Con
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

2.RandomForestClassifier

```
from sklearn.ensemble import RandomForestClassifier
r=RandomForestClassifier(n_estimators=150,max_depth=10)
r=r.fit(X_train,Y_train)
print('Random Forest predicted training score',r.score(X_train,Y_train)*100)
print('Random Forest predicted test score',r.score(X_test,Y_test)*100)
```

```
Random Forest predicted training score 84.27062596770625
Random Forest predicted test score 84.04290611522724
```

3.XGBoost

```
from xgboost import XGBClassifier
x=XGBClassifier()
x.fit(X_train,Y_train)
print('XGBoost predicated training score',x.score(X_train,Y_train)*100)
print('XGBoost predicated test score',x.score(X_test,Y_test)*100)
```

```
XGBoost predicated training score 85.34616235346162
XGBoost predicated test score 83.7996240185779
```

Start coding or generate with AI.

Using Piepeline to ease the deployment and make it reusable

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```
df=pd.read_excel('/content/bank_data.xlsx')
```

```
df=df.drop(df.iloc[:,8:17],axis=1)
```

```
df.sample(3)
```

	age	job	marital	education	default	balance	housing	loan	loan stats	L
4466	40	blue-collar	married	secondary	no	-438	no	no	0	
4316	41	blue-collar	married	primary	no	194	yes	no	0	
40249	40	technician	married	secondary	no	4441	yes	no	0	

```
y = df['loan'].replace({'no': 0, 'yes': 1})
```

```
/tmp/ipykernel_42822/446897862.py:1: FutureWarning: Downcasting behavior in `repla
  y = df['loan'].replace({'no': 0, 'yes': 1})
```

```
y = df['loan'].replace({'no': 0, 'yes': 1})
x = df.drop(['loan'], axis=1)
```

```
/tmp/ipykernel_42822/4237197413.py:1: FutureWarning: Downcasting behavior in `repl
  y = df['loan'].replace({'no': 0, 'yes': 1})
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=.2,random_state=43)
```

```
x_train.shape,x_test.shape
```

```
((36168, 11), (9043, 11))
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

ct = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown='ignore'), ['job']),
    ('ord', OrdinalEncoder(categories=[
        ['divorced', 'married', 'single'],
        ['secondary','primary','tertiary','unknown'],
        ['no','yes'],
        ['no','yes']
    ]), ['marital', 'education', 'default', 'housing']),
 ('num', MinMaxScaler(), ['age', 'balance', 'loan stats', 'Loan_period'])
```

```
], remainder='passthrough')
```

```
df.sample()
```

	age	job	marital	education	default	balance	housing	loan	loan stats	Lo
29490	30	services	married	secondary	no	292	yes	yes	0	

```
from sklearn.feature_selection import SelectKBest, chi2
ct4=SelectKBest(score_func=chi2,k=12)
```

1.RandomForestClassifier

```
from sklearn.ensemble import RandomForestClassifier
rf=RandomForestClassifier()
```

2.LogisticRegression

```
from sklearn.linear_model import LogisticRegression
lgr=LogisticRegression()
```

3.XGBClassifier

```
from xgboost import XGBClassifier
xgb=XGBClassifier()
```

Creating pipeline for the models

```
pipe=Pipeline([
    ('ct',ct),
    ('ct4',ct4),
    ('lgr',lgr)
])
```

```
pipe=pipe.fit(x_train,y_train)
```

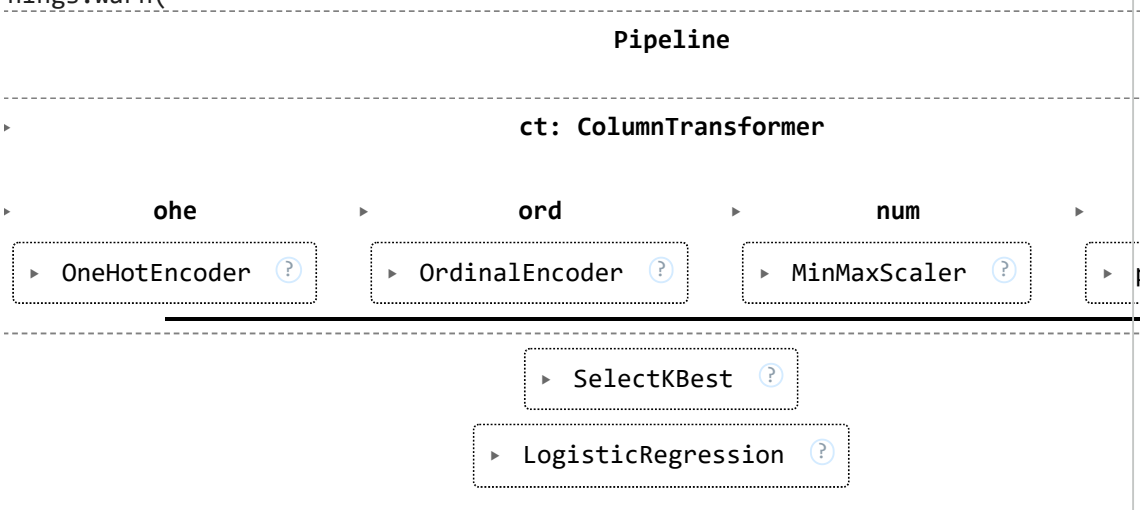
```
from sklearn import set_config
set_config(display='diagram')
```

Pipeline workflow

```
pipe
```

```
local/lib/python3.12/dist-packages/sklearn/compose/_column_transformer.py:1667
format of the columns of the 'remainder' transformer in ColumnTransformer.trans
e moment the remainder columns are stored as indices (of type int). With the s
e the new behavior now and suppress this warning, use ColumnTransformer(force_

nings.warn(
```




```
y_pred=pipe.predict(x_test)
```

```
print('Logistic Regression predicted training score',pipe.score(x_train,y_train)*
print('Logistic Regression predicted test score',pipe.score(x_test,y_test)*100)
```

```
Logistic Regression predicted training score 83.82824596328246
Logistic Regression predicted test score 84.5184120314055
```

Importing pipeline using pickel library

```
import pickle
pickle.dump(pipe,open('pipe.pkl','wb'))
```

Start coding or generate with AI.